Information and Software Technology 196 (2026) 108150

Contents lists available at ScienceDirect

Information and Software Technology

journal homepage: www.elsevier.com/locate/infsof

Towards metamorphic testing with LLM-based workflows: Metamorphic relation inference and follow-up test case generation

Pablo C. Cañizares [a], Pablo Gómez-Abajo [b], Esther Guerra [b], Juan de Lara [b,*]

[a] Department of Computer Science Universidad Complutense de Madrid, Spain
[b] Department of Computer Science, Universidad Autónoma de Madrid, Spain

ARTICLE INFO

Dataset link: https://github.com/AI4MT/AI4MT-experiment

Keywords:
Metamorphic testing
Conversational assistants
Large language models
AI-based testing
Model-driven engineering

ABSTRACT

Context: Metamorphic testing (MT) is a well-known approach to address the oracle problem in software testing. It does so by using expected relations – called metamorphic relations (MRs) – between the inputs and outputs of multiple system executions as test oracles. This approach involves identifying meaningful MRs for the domain, defining them in an executable language, and creating suitable system inputs to check their satisfaction. However, these tasks are manual and demand substantial effort from testers.
Objective: To reduce the effort of applying MT, we propose automating some tasks in the MT process with intelligent assistants, focussing on MR inference and generation of follow-up test cases.
Methods: We propose a taxonomy of MT tasks amenable to LLM-based assistance, and an extensible architecture for their automation. The MT tasks are accessible via a conversational assistant, and may encompass validation and fixing cycles to improve the quality of the assistance. We have integrated the architecture with the MT framework Gotten.
Results: We have evaluated our approach on the two considered MT tasks across three domains (data centres, autonomous vehicles, finite automata). We found the assistant effective at generating new MRs (between 2 and 6 depending on the domain) and follow-up test cases (with over 99.6% correctness). We also measured diversity and redundancy of the generated MRs, the benefits of fixing cycles on different LLMs when generating follow-up test cases, and performance degradation and computational cost as input complexity increases.
Conclusions: Our architecture effectively automates two central MT tasks, namely, inferring MRs and generating follow-up test cases, showing potential to reduce the effort from MT practitioners and to automate other MT tasks.

1. Introduction

Software needs to be reliable to be useful, and testing is a common technique to ensure a proper level of software quality. However, there are systems challenging to test due to the *oracle* problem, i.e., determining their correct output for a given input is difficult [1]. Systems that are hard-to-test include machine learning systems, autonomous driving systems, compilers, image/signal processing software, scientific computing applications, and simulators, among others [2].

Metamorphic testing (MT) [3–5] attacks the previous problem by using relations between the system inputs and their expected outputs as oracle. For example, if the goal is testing the *sin* trigonometric function, one may define the relation $-sin(x) = sin(-x)$. According to this *metamorphic relation* (MR), given two inputs x_1 and x_2 such that $x_1 = -x_2$, the function outputs should be related as $-sin(x_1) = sin(x_2)$. In this setting, a *seed* test case is any valid system input, while a *source* test case is a seed test case used to generate *follow-up* test cases ("*follow-ups*" for short). Throughout this paper, we use source test case and seed test case interchangeably when there is no ambiguity. In the *sin* example, a source test case consists of a value for x_1, and a follow-up (a value for x_2) can be generated by negating x_1. These pairs of source and follow-up test cases (values for x_1 and x_2 in the example) can be used to check the MRs and search for software defects.

Even if useful, MT requires expert knowledge to identify meaningful MRs for the given domain, specify them in an executable language, check their correctness, optimise them, and generate follow-ups. Some tools to automate MT have been proposed, supporting the definition and check of MRs, or test case generation [6,7]. However, they generally lack intelligent mechanisms to assist testers throughout the MT process, especially (though not exclusively) in tasks that require domain knowledge.

* Corresponding author.
E-mail addresses: pablocc@ucm.es (P.C. Cañizares), Pablo.GomezA@uam.es (P. Gómez-Abajo), Esther.Guerra@uam.es (E. Guerra), Juan.deLara@uam.es (J.d. Lara).

https://doi.org/10.1016/j.infsof.2026.108150
Received 8 December 2025; Received in revised form 9 April 2026; Accepted 10 April 2026
Available online 13 April 2026
0950-5849/© 2026 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

The recent advances in generative Artificial Intelligence (AI) and Large Language Models (LLMs) have prompted their use to assist in many disciplines. Software testing is no exception to this trend [8]. However, while LLMs have been applied to testing tasks, such as test case generation [9,10], test oracle inference [11] and test maintenance [12], their application to MT has barely been explored. Although a few proposals address individual MT tasks, to our knowledge, no domain-agnostic LLM-based architecture has been proposed to encompass the entire MT process.

To fill this gap, we first propose a taxonomy of MT tasks that are amenable to LLM-based assistance. Second, we introduce an extensible AI-based architecture designed to provide reliable assistance for MT tasks. The architecture allows configuring pipelines of assistive tasks, each optionally followed by validation and fixing stages aimed to improve the quality of the assistance. For example, an assistive task that generates follow-ups could be paired with validators that check the syntactic correctness of the generated test cases and confirm they are valid follow-ups of the given source test case and MR; as well as with fixers that automatically amend the incorrect follow-ups when the validations fail. For space limitations and to maintain focus, in this paper, we concentrate on two representative MT tasks that are central to the MT process: inference of MRs, and generation of follow-ups. We have integrated our assistive architecture with the MT framework Gotten [6], which permits defining and executing MRs using a domain-specific language (DSL). Gotten now offers access to the assistance via a conversational interface.

To assess our proposal, we conducted evaluations for the two MT tasks on three domains (data centres, autonomous vehicles, and finite automata). For MR inference, we confirmed that our assistant is able to find a diverse set of new MRs (it found between 2 and 6 per domain) in a reasonable time (9.97 s per MR in average), and providing adequate context in the prompt is important. For follow-up generation, we obtained the best results (over 99.6% correctness) when using LLMs based on reasoning (*o4-mini* and *gpt-5-mini*). The amendment loops improved the effectiveness of follow-up generation by 1.35 to 35.57 percentage points, though with a slight impact on the computation cost. Moreover, increasing the size of the source test cases caused marginal or no performance degradation for reasoning LLMs, but led to drops of up to 46.5 percentage points for other LLMs. These results show the potential of LLM-based workflows for MT automation, and may serve as a guide for MT researchers to build AI assistants.

Paper organisation. First, Section 2 reviews related research on AI assistance for MT. Next, Section 3 presents our proposal, and Section 4 describes tool support. Section 5 reports on the evaluation. Finally, Section 6 concludes.

2. State of the art

Researchers have applied AI to improve various phases of MT. Next, we review AI-based approaches addressing the two tasks targeted in this paper: *MR inference* (Section 2.1) and *follow-up generation* (Section 2.2).

2.1. MR inference

The first works on MT created MRs ad-hoc, which is challenging and requires expertise. This led to proposals for systematic MR generation which, according to [13], can be classified as pattern-based, search-based, and AI-based.

Pattern-based approaches rely on pre-defined patterns, templates or *general MRs* [14] to derive multiple concrete MRs for specific types of systems.

Search-based approaches reformulate MR generation as an optimisation problem. For instance, GenMorph [15] generates MRs for Java methods by means of an evolutionary algorithm guided by two fitness functions: the number of false alarms and the number of missed faults for the generated MRs.

In this paper, we are interested on AI-based approaches. Some of them [16] focus solely on the early stage of variable discovery, using LLMs to extract relevant input/output variables from user manuals as an initial step in MR design. Instead, more comprehensive approaches use LLMs to generate complete MRs from system requirements. For example, in the autonomous driving domain, Zhang et al. [17] use ChatGPT to generate MRs expressed in NL, and therefore not directly executable. Subsequently, the authors refined their work [18], generating MRs from user-provided specifications via a GPT model customised with MR patterns (MRPs). The resulting MRs are refined by MT experts and fed into a test harness that automates test creation and execution, and generates follow-ups by modifying parameters of existing scenarios (i.e., follow-up generation is not LLM-based). Luu et al. [19] use GPT and zero-shot prompting to generate MRs for a given program. The work is preliminary, and most generated MRs are vague or incorrect, particularly for systems not previously tested with MT. Closer to our approach, Shin et al. [20] derive executable MRs by using few-shot prompting, where the LLM is provided the requirements of the system under test, and the language the MRs should be written in (SMRL). In addition, we also consider validation and fixing cycles (to avoid generating incorrect or hallucinated MRs) and cover other MT tasks (e.g., follow-up generation).

Other proposals for MR inference are not pattern-based, search-based or AI-based. For instance, Pedram and Labiche [21] use category partition functional testing to suggest MRs, and Xu et al. [22] apply static analysis to identify developer-written test cases that implicitly encode MRs, from which new MRs and tests are synthesised. MR composition is another way to generate MRs by combining existing ones [23], which reduces the number of follow-ups and the testing costs. While some studies suggest that composite MRs may have lower fault detection capabilities than their components [24], researchers have mitigated this by using genetic algorithms to obtain optimal composite MRs [25], or by identifying the characteristics individual MRs of a composite MR should have to preserve their fault detection capability [26].

Contribution: Table 1(a) summarises the main features of the surveyed LLM-based approaches to MR inference. Some approaches target a specific domain (e.g., autonomous driving), which limits their reusability, whereas others (including ours) are domain-agnostic. Most approaches infer non-executable MRs, making their application time-consuming and error-prone. The only exceptions are the work in [20], which derives executable SMRL rules, and our proposal, which infers executable Gotten rules. Finally, compared to all existing LLM-based approaches, ours is the only one incorporating validation and fixing cycles (which improves the precision of the inferred MRs, cf. Section 5) and defines a generic workflow that supports other MT tasks.

2.2. Follow-up generation

LLMs have been employed to synthesise follow-ups for specific MRs. However, most works concentrate on MT of LLMs. For example, METAL [30] is an MT framework for analysing quality properties of LLMs, which permits generating concrete MRs (akin to follow-ups) from five predefined MR templates by applying perturbation functions and LLM-based text perturbation prompts like "replace a maximum of three synonyms for the above text". The authors propose a quality metric to detect low-quality and incorrectly generated perturbed texts, though these are not automatically fixed. Similarly, Cho et al. [28] define a set of 36 MRs for LLMs, where input test cases are natural language (NL) sentences, and follow-ups are generated using functions (for simple MRs) or few-shot prompting with LLMs (for complex MRs that require paraphrasing). Since the input cases are NL sentences, using LLMs permits generating textual follow-ups that are grammatically correct. However, in their experiments, some follow-ups were either

Table 1
Comparison of LLM-based approaches to MR inference and follow-up generation.

(a) MR Inference

Proposal	Infers	Executable MR?	Technique	Domain	Validation & Fixing Cycle?	Other MT tasks?
Luu et al. [19]	MRs	No	Zero-shot prompting	Any (code)	No	No
Shin et al. [20]	MRs	Yes (SMRL)	Few-shot prompting	Any (code)	No	No
Tsigkanos et al. [16]	I/O features	No	Few-shot prompting	Scientific sw	No	No
Zhang et al. [17]	MRs	No	ChatGPT	Auton. driving	No	No
Zhang et al. [18]	MRs	No	GPT + MR patterns	Auton. driving	No	Follow-up gen.
Our approach	MRs	Yes (GOTTEN)	Few-shot prompting	Any (model)	Yes	Yes: Section 3.2

(b) Follow-up Generation

Proposal	Infers	Fixed MRs or Follow-up Opers?	Technique	Domain	Validation & Fixing Cycle?	Other MT tasks?
Chan et al. [27]	NL text	Yes (3 MRs, 7 ops)	Programmatic, LLM	LLMs	No	No
Cho et al. [28]	NL text	Yes (36 MRs, 36 ops)	Programmatic, LLM	LLMs	No	No
Guo et al. [29]	Question sequence	Yes (4 MRs, 5 ops)	Programmatic	LLMs	No	No
Hyun et al. [30]	NL text	Yes (5 MRs, 13 ops)	Programmatic, LLM	LLMs	Validation (metric)	No
Zhang et al. [18]	Param. values	Yes (3 MRPs, 1op/MR)	Programmatic	Auton. driving	No	MR inference
Our approach	Model	No	LLM	Any (model)	Yes	Yes: Section 3.2

too similar or too different from the source test cases, resulting in a non-negligible ratio of false positives during MR evaluation. Chan et al. [27] analyse the stability of LLMs by using three symmetric, semantic-preserving MRs (e.g., case transformation of prompts), and also generate follow-ups of NL prompts using LLMs and paraphrasing tools.

While the previous works focus on single-turn conversations, MORTAR [29] tackles multi-turn interactions. It defines four MRs and five dialogue-level perturbations to generate follow-ups influencing the question sequence, the conversational context and the answerability of questions in perturbed test cases.

Contribution: Table 1(b) summarises the analysed approaches. All target a single domain, frequently LLMs and NLP tasks, and define highly specialised testing processes with fixed sets of MRs and operations to generate follow-ups. In consequence, these approaches are not easily transferable to other domains. Instead, our proposal uses LLMs to generate follow-ups in arbitrary domains, the set of MRs and follow-up generation operations is not fixed a-priori, and source test cases are not constrained to be text in NL. Moreover, unlike existing LLM-based follow-up generation approaches, ours incorporates fixing cycles, and its workflow is applicable to other MT tasks, as Section 3.3 shows.

3. AI-assisted metamorphic testing

This section describes our framework for AI-assisted MT. Section 3.1 starts by detailing the elements required from an MT environment to enable the integration of our AI assistance. Section 3.2 defines a taxonomy of MT tasks suitable for AI assistance. Finally, Section 3.3 proposes an LLM-based workflow for MT assistance tasks that incorporates quality improvement mechanisms and can be used via a conversational interface.

As a running example, we consider the MT of a program to calculate the area of triangles and classify them based on the length of their sides (scalene, isosceles or equilateral). This toy example is often used to illustrate testing techniques, including MT [2,31]. We use it to explain the concepts due its simplicity, but Section 5 presents a full-fledged evaluation with more complex examples.

3.1. Enabling AI assistance in MT environments

Our approach to augment an MT environment with AI-based assistance requires the environment to provide the following elements:

- A schema describing the syntax and correctness constraints of the inputs of the *system-under-test* (SuT). Any approach can be used to define the space of inputs, including meta-models [6], grammars [32], JSON/XML schemas [33], or data types [34]. In this paper, we consider that SuT inputs are models conformant to their meta-models.
- A specification of the relevant features that can be extracted from the SuT outputs. This permits their use in MRs.
- An executable language for expressing MRs. Although this can be any general-purpose language (e.g., Python), in this paper, we use a DSL called GOTTEN that permits defining MRs using a declarative, OCL-based [35] syntax. MRs in GOTTEN are logical implications whose left-hand side relates features derived from the SuT inputs, and whose right-hand side relates features derived from the SuT outputs.
- An execution environment for MT, able to execute the SuT on valid inputs and follow-ups, and check the satisfaction of MRs. If an MR is not satisfied, then it may signal a failure (or, alternatively, the MR may be incorrect).

Example. Fig. 1 illustrates these elements for the running example. The meta-model in the top-left corner captures the schema of the inputs: triangles with the length of their three sides. The OCL constraint ensures that the lengths are positive, thus modelling an assumption of the SuT. The input models *m1* and *m2* in the bottom-left corner are both valid instances of the meta-model. The right of the figure shows the definition of an MR in the GOTTEN DSL, which in turn uses OCL (e.g., types like Sequence, and operations like asSet). First, lines 1–4 declare one *input feature*, that is, a value derived from the input model. The feature, called Sizes, returns a collection with the lengths of the triangle sides. Lines 6–8 declare an *output feature* called TriType, which will be obtained from the SuT outputs. It encodes the type of triangle returned (0: Not a triangle, 1: Equilateral, 2: Isosceles, 3: Scalene). Finally, lines 11–13 declare an MR named MR1. This states that, if two input models *m1* and *m2* have the same side lengths, then the output of the SuT (TriType) for both models should be identical. Under this MR, models *m1* and *m2* in the figure are follow-ups of each other. An MT environment can therefore execute the SuT on both models, and use the MR to assess that their outputs coincide.

3.2. A taxonomy of AI-assisted MT tasks

We classify assistive tasks for MT in three categories, depending on whether they target MRs (their construction and comprehension), the MT process (generating and understanding test cases, exploiting the MT

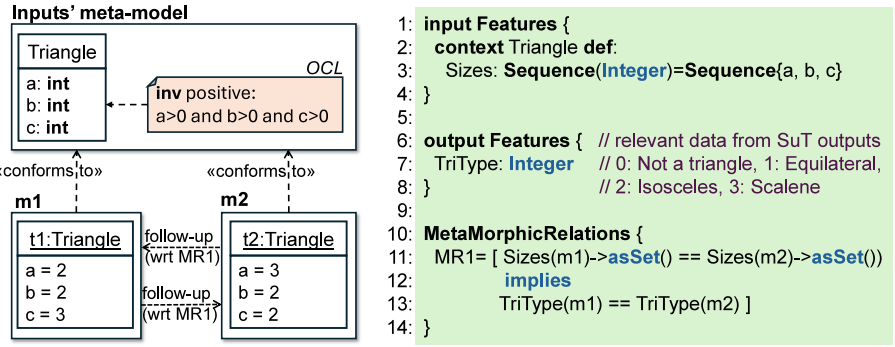


Fig. 1. Elements of the running example: input meta-model, MR using the Gotten DSL, and two input models (each one a follow-up of the other).

results for fault localisation), and the MT infrastructure (setting up the infrastructure described in Section 3.1 that enables MT on a SuT). Next, we identify concrete tasks in each category.

1. **Metamorphic relations (MRs)**. These tasks assist in the creation, refinement and comprehension of MRs.

   - *MR inference*. This task creates a new MR, preferably one that analyses uncovered aspects of the SuT. Possible inputs to the task include: (i) a description of the functionalities, inputs and outputs of the SuT; (ii) a textual description of the MR to be created, if known in advance; (iii) technical documents describing the SuT, or relevant excerpts of its implementation code; (iv) other MRs of the SuT (to provide information of aspects already covered). The output of the task is an executable MR expressed in a general-purpose language or a DSL.
   - *MR composition*. Composing MRs may increase the cost-effectiveness of MT by reducing the number of follow-ups [23,25,26]. Thus, this assistive task combines existing MRs into new ones. It can be seen as a special case of MR inference, where the input is a set of existing MRs, and the output MR does not analyse uncovered aspects of a SuT but meaningfully combines existing MRs for optimising the testing process.
   - *MR optimisation*. This task reduces the complexity of existing MRs while preserving their semantics. MR optimisations may seek to improve performance or comprehensibility. In the first case, unnecessary terms are removed, and the computation logic may change to make the MT process more efficient. In the second case, the MR is reformulated to make it easier for humans to understand.
   - *MR repair*. The assistance in this case fixes problems in an existing MR. Problems may be syntactic (easier to fix) or semantic (e.g., an MR that fails on a correct SuT for some input–output pairs, or an MR with typing errors that make the MR evaluation crash at runtime). The input to this task comprises the faulty MR and the input–output pair that causes the failure. In case of runtime errors, the input also includes the error trace.
   - *MR comprehension*. This task offers NL explanations of existing MRs, defined in general-purpose languages or DSLs.

2. **MT process**. These assistive tasks target the MT process, from automating the creation of SuT inputs and follow-ups, to facilitating the interpretation and exploitation of MT results.

   - *Generation of seed test cases*. This task generates suitable inputs for the SuT, to be used as test cases. It receives the schema of the inputs, and optionally, an NL description of the test cases (e.g., "*triangles with very dissimilar side lengths*"). The output can be one seed test case, or several ones attending to specified criteria like diversity or coverage (e.g., of the input space, of the SuT).
   - *Generation of follow-ups*. This task generates follow-ups that enable the evaluation of an MR. Hence, it receives an MR and a source test case as input. Like in the previous task, the output can be one or several follow-ups that fulfil certain generation criteria.
   - *MT result comprehension*. This task explains, in NL, the outcomes of an MT process. It can operate at two granularity levels: individual MRs, and the entire MT process. For an MR, the task explains the results of evaluating that MR over an input pair, whether it is satisfied, not satisfied (detailing the reasons for failure), or not evaluated because the inputs do not satisfy the MR precondition (explaining why). When considering an entire MT process comprising multiple MRs and test cases, the task summarises the results and highlights the most and least successful MRs.
   - *Fault localisation*. Whenever an MR is not satisfied, this task helps locating the source of the problem in the SuT.

3. **MT infrastructure**. These tasks help in creating the infrastructure to perform MT.

   - *Inference of input schema*. This task assists in creating a schema that describes the input data of the SuT. For this purpose, it can receive a description of the input data, existing test cases (if available), and the SuT. The output is a schema.
   - *Inference of relevant output features*. This task helps identify useful features that can be computed from the SuT outputs for their use within MRs. It can receive existing outputs (if available) and the SuT, and suggests potential features along with explanations of their suitability (e.g., which properties it could help assess).
   - *Completing execution infrastructure*. Here, we encompass any activities helping in completing the infrastructure needed to execute the MT process on a specific MT framework. In the case of Gotten, there would be an assistive task to convert from the input format of the SuT to the input schema, and another to extract the relevant output features from the SuT outputs.

Example. Assume a tester has defined the following MR for the running example: `Sizes(m1)->includesAll(Sizes(m2)) implies TriType(m1) == TriType(m2)`. There is a subtle error, which manifests when the second triangle has sides with the same length. When asking the assistant for help (task *MR repair*), it proposes changing the MR to: `Sizes(m1)->asSet() == Sizes(m2)->asSet() implies TriType(m1) == TriType(m2)`. This MR (shown in Fig. 1) is now correct, since it converts sequences to sets, and performs equality testing on the sets.

3.3. An LLM-based assistance framework for MT

Next, we propose a general workflow for MT assistive tasks that incorporates quality improvement mechanisms (Section 3.3.1), enables

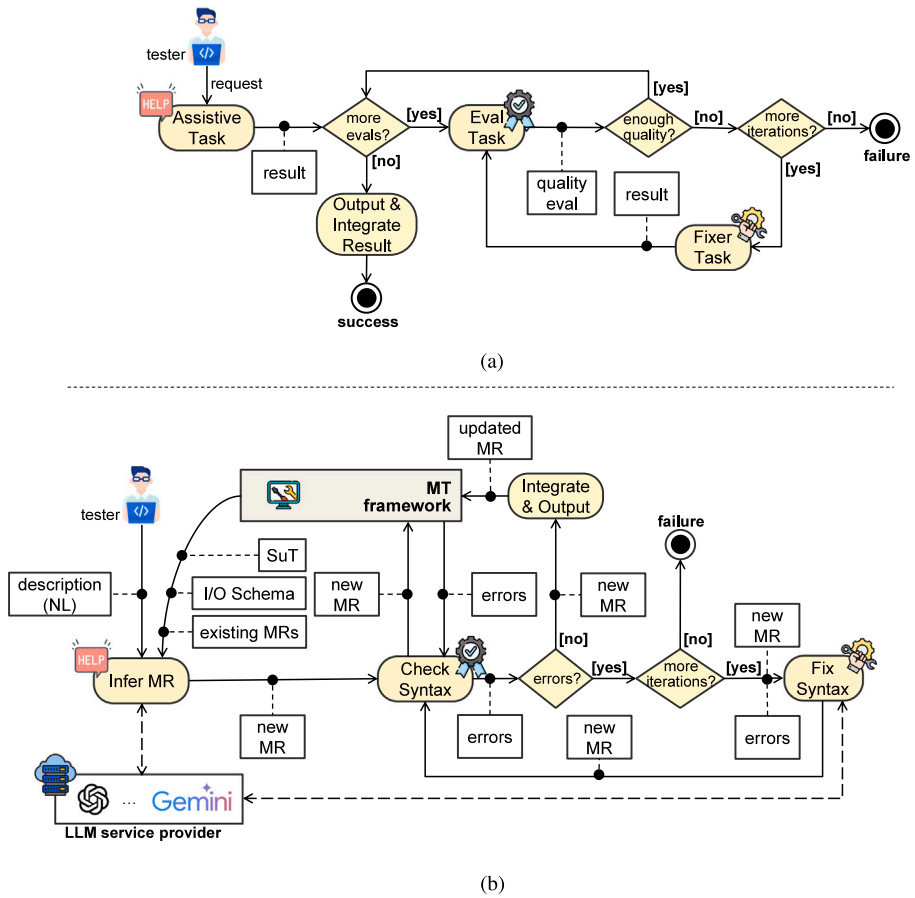


Fig. 2. (a) Generic workflow for LLM-assisted MT tasks. (b) Specific workflow for LLM-assisted MR inference.

sequences of assistive tasks to handle complex cases (Section 3.3.2), and can be accessed via a conversational assistant to simplify its usage (Section 3.3.3).

3.3.1. A quality-aware workflow for MT assistive tasks

The simplest approach to implement LLM-based support for the assistive tasks in our taxonomy consists in designing task-specific prompts that include the necessary information to accomplish the task (e.g., existing MRs, input schema). However, this may fall short since LLMs may produce erroneous outputs due to hallucinations, or non-optimal responses. Therefore, to enhance the quality of the assistance, we propose a quality-aware workflow that incorporates a validation and improvement cycle on the LLM responses. Fig. 2(a) shows its scheme.

First, the tester requests help from an assistive task (e.g., inferring an MR, generating a follow-up), which internally prompts an LLM to obtain a *result*. Assistive tasks can define improvement pipelines made of sequences of pairs of *evaluation* and *fixer* tasks. Thus, the defined improvement pipeline is activated on the obtained result. The first evaluation task in the pipeline is invoked, emitting a quality evaluation report comprising a quality rank (where 0 is top quality) and error messages (if any). If the quality rank meets a given threshold (e.g., it is 0), the pipeline assesses the next evaluation task. Otherwise, the fixer task paired with the evaluation task is applied. Fixer tasks can be LLM-based or not; those that are, typically take the evaluation report and the LLM result, and generate a new result which is evaluated again for quality and fixed if needed. This iteration is repeated until the result meets the quality threshold, a maximum number of iterations is reached, or subsequent iterations do not improve the quality. If an acceptable result is finally obtained (i.e., all evaluation tasks pass successfully), the assistant communicates the improved result to the tester, and integrates it within the MT project.

This generic workflow needs to be specialised per assistive task. For example, Fig. 2(b) shows the specialised workflow for a task that infers an executable MR out of NL. It includes an improvement pipeline with an evaluation task that performs syntax checking of the inferred MR, and a fixer task that fixes the MR when it is syntactically invalid. Instead, an assistive task for generating follow-ups for a source test case and an MR can define an improvement pipeline with two validation steps: the first one checks the syntax of the follow-up (i.e., it conforms to the input schema), and the second one its semantics (i.e., it is a follow-up). A follow-up is semantically valid if the left-hand side of the MR holds, and the "*frame condition*" is satisfied (i.e., the source test case and the follow-up only differ in elements of the MR's left-hand side, and nothing else is modified [6]).

3.3.2. Combination of LLM-based assistive workflows

LLM-based assistive workflows can be combined to support more complex MT assistance processes. For example, the workflow in Fig. 2(b) only checks the syntax of the inferred MR. Instead, the tester may also want to assess its semantic correctness by generating follow-ups and running the MT process. This entails sequentially executing two assistive tasks (MR inference, generation of follow-ups) and one functionality of the MT framework (performing MT with the inferred MR and generated follow-ups). Our framework supports defining sequences of both assistive tasks and MT framework functionalities, which communicate via a shared state (e.g., the first task generates an MR, which the second task consumes).

Generally, tasks that yield an MR – like repairing, composing or optimising an MR – would need a pipeline like the one described for inference. The pipeline for MR optimisation may also include a validation task comparing the results of the execution of both MRs on suites of source and follow-up test cases. Other tasks producing artefacts (metamodels, output features, seed test cases) require syntactical validation.

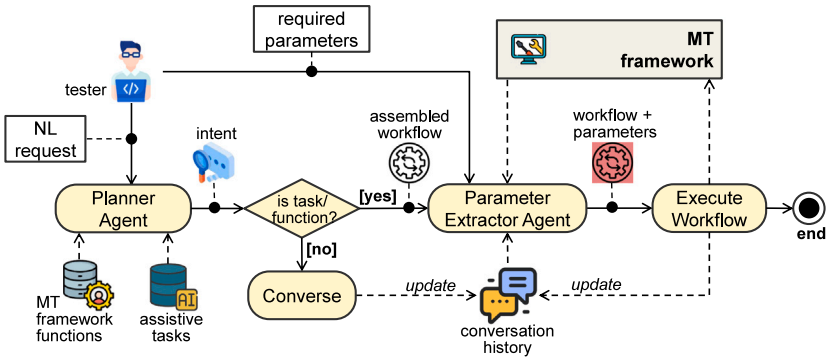


Fig. 3. Orchestration of MT assistance workflows.

3.3.3. Conversational access to MT assistance

The access to assistive tasks, and the configuration of improvement pipelines and tasks' sequences, need to be offered to testers via useable interfaces. Our proposal considers two interaction options: GUI commands, and a conversational interface. The latter maintains a conversation history of recent tester–assistant interactions, and uses LLM-based agents to interpret NL requests and collect the information needed to perform tasks.

Fig. 3 shows the working scheme of the conversational interface. When the tester issues an NL request, a *Planner Agent* identifies the tester's intent. For this purpose, it prompts an LLM to classify the request into one of four possible intents: invoking an assistive task (e.g., inferring a new MR), executing a function of the MT framework (e.g., running a test case on the SuT), assembling a tasks' sequence (e.g., creating an MR and then generating follow-ups for it), or none of them. In the last case, the request and the conversation history are forwarded to an LLM, the response is returned to the tester, and the conversation history is updated. Instead, if the request involves an assistive task or an MT framework function, a *Parameter Extractor Agent* takes care of gathering the parameters needed to execute it (e.g., from the request or the conversation history), asking the tester for any missing information. After collecting all parameters, the system executes the corresponding task.

Example. Suppose the tester says "*I want to generate follow-ups*". The *Planner Agent* would detect the intent "follow-up generation", which is an assistive task. This task requires three parameters: an MR, a source test case, and the number of follow-ups. The *Parameter Extractor Agent* would query the tester for these items and, once obtained, the follow-up generation would be executed.

4. Architecture and tool support

This section describes our supporting architecture for LLM-based MT assistance (Section 4.1), provides details on prompting (Section 4.2), and reports on the integration of the assistive architecture within Gotten (Section 4.3).

4.1. LLM-based assistance architecture for MT

The MT AI assistance architecture has been designed as an object-oriented library. Fig. 4 presents an excerpt, divided in four packages.

Package tasks features a hierarchy of MT tasks (class MTTask), which can be assistive tasks based on AI (AITask, cf. Section 3.2) or functions offered by the MT framework (FrameworkTask). Tasks have an identifier, a description, and input parameters (Parameter) that may or may not have value at any point. Parameter values need to be provided either by the user (Parameter.userProvided), or else, be obtained from the MT framework (via a class implementing the IMTFramework interface in package providers). For example, an assistive task to explain an MR defines four parameters: the MR name, its code, the input schema, and the relevant output features. From these, the user must provide the MR name, and the MT framework provides the other parameters. Tasks can also be assembled into workflows (MTWorkflow) for their sequential execution. For example, the request "*infer a new MR and check its validity*" can be served by a workflow comprising the InferMR assistive task, the FollowUpGeneration assistive task, and the TestForMR framework task.

Package qaPipeline permits defining improvement pipelines (ImprovementPipeline) for AITasks. Each pipeline includes steps (ImprovementStep), each with an evaluation task (IEvalTask) and a fixer task (IFixerTask). In addition, a step has a maximum number of iterations (maxIterations), may restart the pipeline after failing (withRestart), and may terminate if the quality rank does not improve after the last iteration (earlyTermination). Evaluation tasks define method eval, which returns TaskQualityEvaluationResult objects. Evaluation and fixer tasks share a state (EvaluationContext), and may trace their execution by using the singleton ImprovementTrace. The use of an interface for evaluation and fixer tasks enhances flexibility. For example, it permits using MRTasks as fixer tasks, in which case, instead of prompting an LLM to fix discovered errors, the MR task is retried.

Package providers includes the infrastructure to define providers for tasks (ITaskProvider), as well as the singleton AITaskProvider which acts as a repository of AI tasks. To integrate our assistive architecture into MT frameworks, like Gotten, they must implement the IMTFramework interface. This requires providing access to values of potential tasks' parameters (e.g., the current Gotten MT specification, the input schema, the path where the follow-ups are to be generated), and enabling updates from AI tasks (e.g., updating a Gotten specification with a generated MR). Even if Gotten is model-based, our assistive architecture can be integrated with non model-based MT frameworks written in Java. To achieve this, the framework should implement the IMTFramework interface, exposing the supported MT tasks (e.g., executing follow-ups on the SuT) as subclasses of FrameworkTask, and giving access to framework-level information to populate tasks' parameters (e.g., the folder of generated follow-ups). At the implementation level, the prompts of AITasks should also be adapted, e.g., to consider the programming language used to express the MRs.

Package orchestration contains the agents for the conversational interface described in Section 3.3.3. The PlannerAgent serves user requests by means of registered tasks, or workflows of them. It creates a prompt with the description of the registered tasks (MTTask.description), feeds the prompt to an LLM to select those compatible with the user request, and combines them in a workflow if needed. In turn, the ParameterExtractorAgent gathers the value of the parameters required by the selected tasks. It builds a prompt containing the description of these parameters (Parameter.description), and uses an LLM to extract the parameter values already provided in the user request. If some parameter is missing, the agent asks the user for its value.

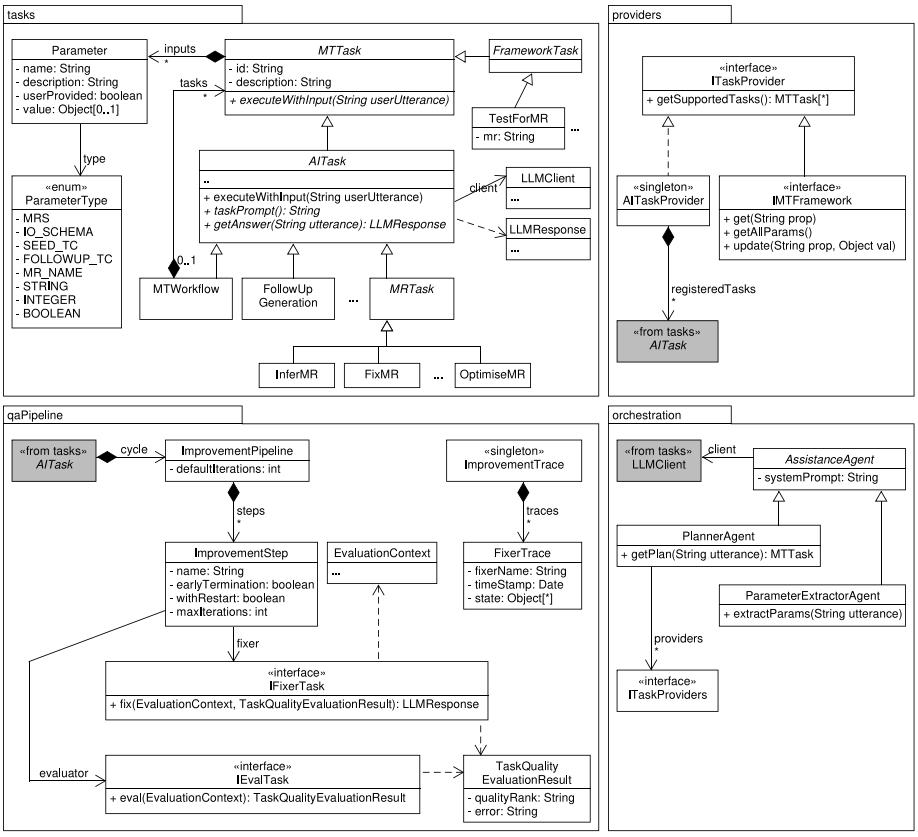


Fig. 4. Architecture of the MT AI assistance.

Technically, we have implemented the architecture in Java to facilitate its integration into our MT framework GOTTEN, as Section 4.3 will show. Other alternative realisations of the architecture could opt for agentic (Python) frameworks like LangGraph[1] or AutoGen.[2]

4.2. Prompting

Next, we provide additional details of the prompts that the PlannerAgent, the ParameterExtractorAgent, and the method taskPrompt of AI tasks use. As an example, we will consider the FollowUpGeneration AI task, whose description is "*Generates follow-up models from a source test case model and an MR*".

The prompt of the PlannerAgent contains the description of all AI tasks (including that of FollowUpGeneration) and asks the LLM to select the tasks dealing with the user utterance. Hence, MTTasks are akin to *Tools* in agent-based frameworks.

Once the intended task (or a set of them) is identified, the prompt of the ParameterExtractorAgent uses the description of the task' parameters to extract their value from the user utterance. For instance, the FollowUpGeneration task has eight parameters. The user must provide two of them (name of MR, and number of follow-ups), and the MT framework provides the other six (path of source test case, path of input meta-model, path of folder where the follow-ups are to be generated, GOTTEN MT specification, and symbolic names of the source test case and follow-ups in the MR, e.g., m1 and m2 in the listing within Fig. 1). Thus, in our example, the prompt only needs to extract the two user-provided parameters.

Finally, for executing the FollowUpGeneration task via an LLM, its method taskPrompt assembles a prompt with the value of the parameters, requesting the generation of a follow-up model in XMI format and conformant to the input meta-model. Then, after obtaining a follow-up, the improvement pipeline is executed. Since the task may need to generate more than one follow-up, it performs a loop. In each iteration, the new prompts include the previous follow-ups, and the LLM is required to produce a different one.

4.3. Integration within GOTTEN

GOTTEN (Generic MDE framework fOr meTamorphic TEstiNg) is a framework to ease the synthesis of MT environments for any domain. Previously, we introduced the framework in [36], evaluated it in [6], and demonstrated it in [37]. The tool is available as an Eclipse plugin at https://g0tten.github.io with examples, tutorials, and video demos.

In this work, we have integrated the presented LLM-based assistance architecture into GOTTEN. This required implementing the interface IMTFramework in Fig. 4. GOTTEN is used via three user roles: the *Application Expert*, the *Domain Expert* and the *MT Tester*. All can benefit from the AI assistance, which is available via a chat and through GUI menus.

First, the *Application Expert* is in charge of creating the meta-model of the SuT inputs in EMF ecore format, and implementing an extension point to allow GOTTEN programmatic access to the SuT and parse the SuT inputs and outputs. This role benefits from the use of the MT infrastructure assistive tasks described in Section 3.2. Second, the *Domain Expert* uses GOTTEN to define MRs using a DSL (cf. Fig. 1) and manually create seed test cases as XMI models. This role benefits from AI tasks related to MRs and the MT process in the taxonomy, such as requesting assistance to create seed test cases with certain features, infer new MRs, or generate follow-ups. Section 5 evaluates the last two tasks.

Finally, the *MT Tester* can define more seed test cases; generate follow-ups from source test cases; evaluate the MRs against the SuT by using the seed test cases and follow-ups; and analyse the test results

[1] https://www.langchain.com/langgraph
[2] https://microsoft.github.io/autogen/stable/index.html

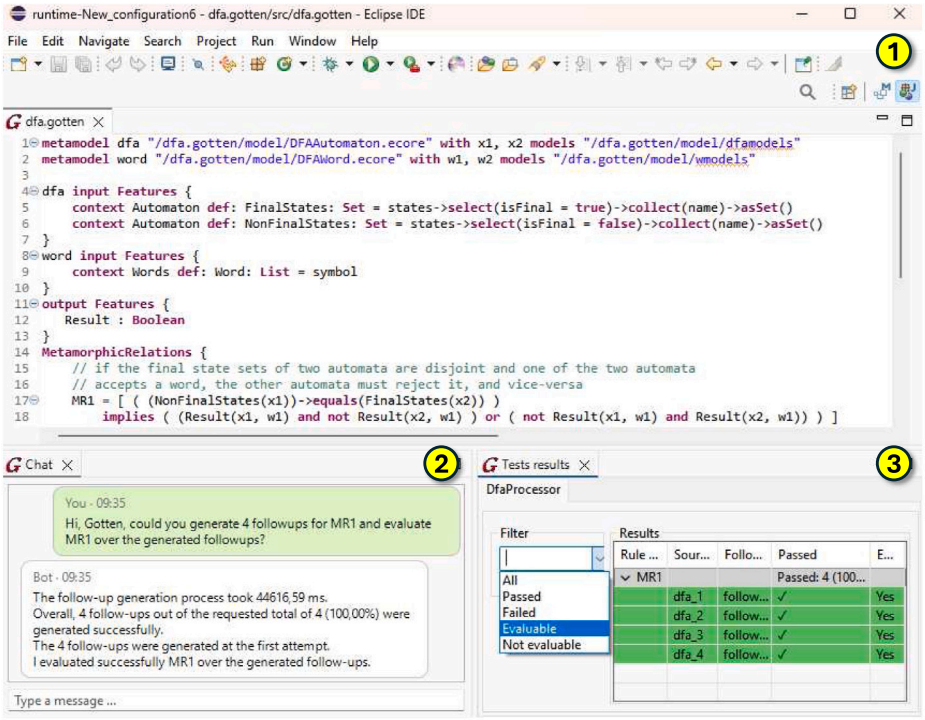


Fig. 5. The Gotten framework in action for MT of automata. (1) Gotten DSL editor. (2) AI chat. (3) MT results view.

in dedicated, interactive reports. This role may use AI assistance in the MT process category of the taxonomy.

Fig. 5 displays a screenshot of Gotten for MT in the domain of automata. The framework has an editor (label 1) to declare the metamodels involved, the input and output features, and the MRs. The view with label 2 is the chat, where users can request assistance in NL. The figure shows the results of requesting the generation of follow-ups (an AI task) and their execution (an MT framework task). The view with label 3 shows the results of the requested execution of MR1. In addition, Gotten features other views, like the follow-ups view, which shows the values of the properties used to evaluate the MRs, and a configuration view to select the LLM used (it currently supports those of OpenAI) and its parameters.

5. Evaluation

This section evaluates our proposal. A thorough assessment of all assistive MT tasks in the taxonomy of Section 3.2 is beyond the scope of a single paper with limited space. Hence, we focus on two representative tasks, MR inference and follow-up generation, as they are central in MT and fall into different taxonomy categories. We aim to answer the following research questions (RQs):

RQ1 How effective is the AI assistance for MR inference?
- **RQ1.1** What is the computational cost?
- **RQ1.2** What is the diversity of the inferred MRs?
- **RQ1.3** How do the inferred MRs compare to human-designed MRs?

RQ2 How effective is the AI assistance for generating follow-ups?
- **RQ2.1** What is the effect of the improvement workflows?
- **RQ2.2** What is the effect of source test case size in correctness?
- **RQ2.3** What is the computational cost?
- **RQ2.4** What is the effect of different LLMs?

The goal of RQ1 is to assess the viability of LLM-based MR inference, and its quality. To address this RQ, we use the assistant to infer new MRs, with an improvement workflow comprising two steps: (i) syntactic validation of the inferred MRs, and (ii) semantic validation of the MRs by executing the MT process with them. In particular, we analyse the effectiveness of four prompt configurations for MR inference, measuring not only the number of inferred MRs but also computational cost (RQ1.1) and MR diversity (RQ1.2), and comparing the complexity of inferred MRs w.r.t. human-designed MRs (RQ1.3).

For RQ2, we evaluate the feasibility of LLM-based follow-up generation, the utility of the improvement workflows, the applicability to source test cases of increasing sizes, and the generalisability to different LLMs. To this aim, we use the assistant to generate new follow-ups, with an improvement workflow with two steps: (i) syntactic validation of the follow-ups, and (ii) semantic validation by checking the MRs on the source test case and follow-ups, and checking the frame condition on the follow-ups. To understand the effect of the improvement pipeline (RQ2.1), we measure the correctness achieved when varying the number of improvement iterations. To understand the effect of source test case size in correctness (RQ2.2) and computational cost (RQ2.3), we use source test cases with increasing sizes. Finally, we analyse the performance and trade-offs of 5 LLMs for follow-up generation (RQ2.4).

Next, Section 5.1 describes the case studies used to answer the RQs, Sections 5.2 and 5.3 evaluate the RQs, and Section 5.4 discusses threats to validity. The data of the experiments is available at https://github.com/AI4MT/AI4MT-experiment.

5.1. Case studies

We consider three case studies in different domains – built using Gotten – to ensure proper experimentation in diverse settings:

- **Data Centres.** This case study, taken from [6], considers models of data centres describing their topology (networks, nodes, racks, boards, switches) and properties (e.g., latency, bandwidth, CPU

Table 2
Summary of experiment case studies.

Case	MM size			Gotten specification size		
	Classes	Attribs.	Refs.	Input Feats.	Output Feats.	MRs
Data Centres [6]	6	8	6	6	2	5
Autonomous Vehicles [38]	5	21	3	3	3	3
Automata	4	5	8	2	1	2
Words	1	2	1	1	1	2

cores, CPU speed, memory speed, disk speed). These models are used with simulators like CloudSim,[3] to measure features like time and energy taken when processing given workloads. The case study considers five MRs.

- **Autonomous Vehicles.** This case study, taken from [38], focuses on testing the navigation system of autonomous vehicles, specifically a model published by MathWorks [39]. It considers features such as minimum and maximum speed, number of waypoints in trajectories, and number of obstacles, to assess the performance and robustness of the path-tracking controller. The main metrics considered are total travelled distance, time to destination, and path-tracking error. We consider three MRs of the original case study.
- **Automata.** This case study addresses the testing of simulators for deterministic automata, and of specific automata. We consider four MRs. Two of them test the simulator, and the other two test a specific automaton that is expected to accept $(0+1)^*0$. The source test cases and follow-ups of the first two MRs are automata (a graph-like structure), while the ones of the last two MRs are words (lists of symbols). We present this case study in two parts, called **Automata** and **Words**, since the nature of their MRs and follow-ups differs (the former handles automata, and the latter words).

Table 2 summarises the case studies, showing the size of the metamodel of the SuT inputs, and the size of the Gotten specification (number of input features, output features, and MRs). The detailed data, including the formulation of the MRs in Gotten, are available at https://github.com/AI4MT/AI4MT-experiment.

5.2. RQ1: Effectiveness of MR inference

Experiment design. In this experiment, we conduct an ablation study to analyse how different configurations of the prompt affect the performance of the AI-assisted MR inference task. Starting from a detailed prompt, we progressively remove information from it, and observe whether this affects the number, correctness, diversity and redundancy of the inferred MRs, and the computational cost of the process.

The most detailed prompt includes: (i) two MRs of the *original* Gotten specification for the case study, their associated input features, and the input meta-model; and (ii) two MRs of an *example* Gotten specification from another case study, and their associated input features. The example specification uses another case study to avoid giving the AI task any knowledge on the SuT.

The ablation study is organised into two main settings: *example* and *no-example*. The *example* setting includes the example Gotten specification in the prompt, whereas the *no-example* setting excludes it, requiring the assistant to infer new MRs solely based on the original specification. Within each setting, we explore four prompt variants that remove parts of the available information: *detailed*, which includes the full original specification (both MRs and input features); *noMR*, which removes all MRs from the original specification while keeping the input features; *noIF*, which preserves the original MRs but omits the input features; and *none*, which removes both the original MRs and their input features.

We conducted all experiments using *gpt-4o-mini*.

[3] http://cloudbus.org/cloudsim/

Experiment results. Table 3 summarises the results of the MR inference task, organised per case study (data centres, autonomous vehicles, automata, words), setting (*example*, *no-example*), and configuration (*detailed*, *noMR*, *noIF*, *none*). For each combination, we report the number of original MRs included in the prompt (*IMR*), the total number of inferred MRs (*#MR*), the number of inferred MRs syntactically valid (*SYN*), the number of inferred MRs semantically valid (*SEM*), and the median inference time across five executions, in seconds (*T (s)*).

The assistant was able to derive correct MRs in all case studies. As expected, the impact of the ablation becomes more pronounced as more elements are removed from the prompt. This is reflected in a gradual decrease in the number of syntactically and semantically valid MRs, with semantic validity being the most affected. The *detailed* configuration consistently achieves the best results, yielding between 2 and 6 semantically valid MRs per case study. In contrast, the *none* configuration obtains between 0 and 2 semantically valid MRs. Across configurations, the assistant generated 10, 14, 6 and 4 different MRs for data centres, autonomous vehicles, automata and words, respectively, of which 8, 12, 4 and 2 MRs were not included as examples in the prompt.

Comparing configurations, removing the original MRs (*noMR*) has a stronger negative impact than removing the input features (*noIF*), especially regarding semantic validity. In three of the four cases, the drop from *detailed* to *noMR* is more severe than the drop from *detailed* to *noIF*, which suggests that the assistant primarily exploits the existing MRs as structural templates from which to extrapolate new candidates. In contrast, when removing input features (*noIF*), the assistant still relies on these templates, but it tends to generate simpler MRs. Including an example in the prompt has an effect, too: when it is removed (*no-example*), the assistant performance drops across all configurations, in terms of both the number of derived MRs and their validity.

Next, we analyse in detail the computational cost, diversity and complexity of the inferred MRs.

RQ1.1. Computational cost. As Table 3 shows, inference was fast, with median times between 11.7 and 36.4 s, and an average computation time of 9.97 s per semantically valid MR. In all case studies except words, inference latency was slightly lower in the *example* setting than in the *no-example* one, but the difference is not statistically significant according to the Wilcoxon test.

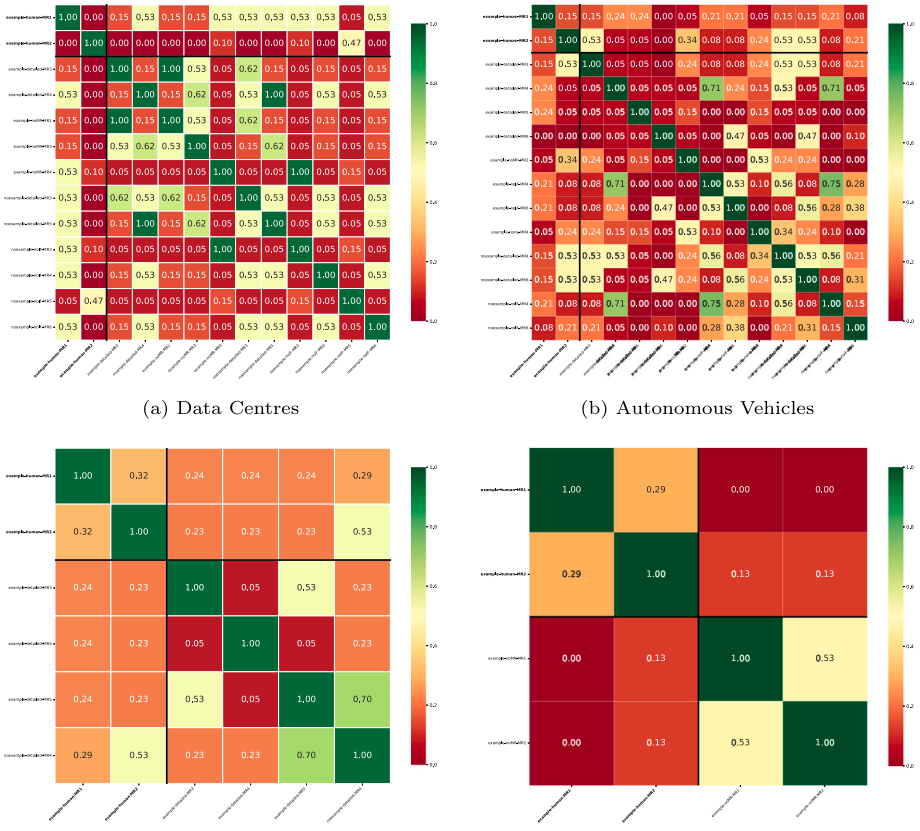
RQ1.2. Diversity of the MRs. To assess MR diversity, we compute pairwise Jaccard similarities[4] [40] using three signatures: (i) a *footprint signature* capturing the set of input and output features used by the MR; (ii) a *signed signature* that augments the footprint with the relational direction of each feature (e.g., >, <, =), thus distinguishing MRs that involve the same variables but perform different operations; and (iii) a *structural signature* that abstracts the MR into its syntactic form (e.g., a single comparison vs a conjunction of conditions), independent of the concrete features involved. Each view produces a normalised Jaccard score in $[0, 1]$, where higher values indicate greater similarity. We then aggregate these scores into a weighted composite similarity:

$$Sim = 0.20 \cdot J_{\text{footprint}} + 0.75 \cdot J_{\text{signed}} + 0.05 \cdot J_{\text{struct}}$$

[4] The Jaccard similarity between two sets is the number of elements they have in common divided by the total number of distinct elements across both sets.

Table 3
Summary of inferred MRs, and their computation time.

Case	Config.	Example					No-example				
		IMR	#MR	SYN	SEM	T (s)	IMR	#MR	SYN	SEM	T (s)
Data Centres	detailed	2	5	5	4	18.7	2	5	5	5	17.7
	noMR	0	4	4	3	17.1	0	6	0	0	20.2
	noIF	2	5	5	4	18.0	2	6	4	2	18.5
	none	0	4	3	2	18.1	0	5	0	0	15.7
Autonomous Vehicles	detailed	2	6	6	6	16.7	2	6	6	5	17.8
	noMR	0	4	4	1	14.5	0	3	0	0	15.8
	noIF	2	6	6	5	14.6	2	6	3	2	15.5
	none	0	4	1	1	11.7	0	3	0	0	13.8
Automata	detailed	2	6	5	5	36.4	2	6	3	3	33.0
	noMR	0	6	6	1	25.9	0	5	0	0	24.3
	noIF	2	6	2	2	23.2	2	6	2	2	24.6
	none	0	5	0	0	14.2	0	6	2	0	13.0
Words	detailed	2	5	2	2	26.5	2	6	2	2	27.0
	noMR	0	4	4	2	14.0	0	5	0	0	14.6
	noIF	2	6	2	2	28.2	2	6	2	2	25.3
	none	0	5	0	0	13.1	0	5	0	0	20.3



(a) Data Centres (b) Autonomous Vehicles

(c) Automata (d) Words

Fig. 6. MR similarity matrices based on Jaccard similarity.

This weighting emphasises semantic similarity (signed signature), while still accounting for feature coverage (footprint) and, to a lesser extent, similar syntactic patterns (structural).

Fig. 6 shows the similarity matrices for the syntactically and semantically valid MRs inferred in the case studies. Each matrix displays the pairwise similarity between MRs based on the composite similarity score defined above.

The similarity of the inferred MRs for data centres is generally low. Across 78 MR pairs, the mean similarity is 0.247 (median 0.15, standard deviation 0.264). Most pairs (43.7%) fall in the [0.0, 0.1) range, and thus have low similarity. However, a 28.2% of pairs lies in the [0.5, 0.7) range, indicating MR pairs moderately similar, and 3 MRs were found to be equal (similarity 1). The MRs for autonomous vehicles follow a similar pattern. Considering 91 MR pairs, the mean similarity is 0.197 (median 0.15, standard deviation 0.199), and most similarity values (46.2%) concentrate in the [0.0, 0.1) range. Regarding automata, the analysis of 15 MR pairs yields a mean similarity of 0.286 (median 0.237, standard deviation 0.05), and the similarity of most pairs (60.0%) is in the range [0.1, 0.3). Finally, the words case considers 6 MR pairs, the mean similarity is 0.179 (median 0.127, standard deviation 0.184), and the most frequent similarity values (50.0%) are in the range [0.1, 0.3). These results indicate that the inferred MRs are largely diverse, although a small subset exhibits moderate similarity (between 0.4 and 0.7).

Table 4
Complexity metrics by case study and source (human-designed vs AI-inferred).

Case	Source	Tree Size			Depth			Volume			Difficulty			Effort		
		avg	min	max	avg	min	max	avg	min	max	avg	min	max	avg	min	max
Data Centres	Human (5 MRs)	11.40	9	13	3.60	3	4	26.14	15.51	33.22	1.60	1.00	2.00	46.07	15.51	66.44
	AI (8 MRs)	9.00	9	9	3.00	3	3	15.31	13.93	15.51	0.94	0.50	1.00	14.44	6.97	15.51
Autonomous Vehicles	Human (3 MRs)	13.00	13	13	4.00	4	4	31.70	31.70	31.70	1.50	1.50	1.50	47.55	47.55	47.55
	AI (12 MRs)	11.33	9	13	3.58	3	4	25.21	15.51	33.22	1.38	1.00	2.00	37.35	15.51	66.44
Automata	Human (2 MRs)	13.50	10	17	4.50	4	5	31.23	18.09	44.38	2.62	1.50	3.75	96.78	27.14	166.42
	AI (4 MRs)	11.00	11	11	4.00	4	4	22.84	22.46	24.00	1.62	1.50	2.00	37.27	33.69	48.00
Words	Human (2 MRs)	16.50	15	18	4.00	4	4	46.81	39.86	53.77	2.00	2.00	2.00	93.64	79.73	107.55
	AI (2 MRs)	12.00	12	12	4.00	4	4	27.00	27.00	27.00	2.50	2.50	2.50	67.50	67.50	67.50

RQ1.3. Comparison with human-designed MRs. Next, we examine whether the AI-inferred MRs have a complexity comparable to that of human-designed MRs. For this purpose, we represent each MR as an abstract syntax tree, and compute structural metrics (tree size, tree depth) and Halstead complexity metrics (volume, difficulty, effort) [41]. Volume describes code size based on the number of operations and operands. Difficulty estimates error proneness, and is proportional to the number of unique operators used. Effort represents understandability, and is computed from volume and effort.

Table 4 shows the metric values per case study. We observe that the inferred MRs have comparable depth but tend to be simpler than human-designed MRs, as reflected by the smaller tree size, volume and effort in all case studies. Depth remains in a narrow range for both inferred and human-designed MRs (typically 3–4), suggesting that the assistant preserves their characteristic nesting structure rather than producing oversimplified MRs. The inferred MRs for data centres are smaller (tree size 9, depth 3) and exhibit substantially lower volume and effort (average effort 46.07 vs 14.44). Interestingly, the human-designed MRs often differ from the inferred ones by including an additional comparison of output features, increasing the number of operands and operators. In autonomous vehicles, the three human-designed MRs are homogeneous (min=max). The inferred MRs show slightly lower average tree size (13 vs 11.33) and effort (47.55 vs 37.35), though some reach or even surpass the effort of the human-designed MRs (maximum effort 47.55 vs 66.44). For automata and words, the results must be interpreted cautiously due to the small number of human-designed MRs, which makes a single highly-complex human-designed MR for automata (depth=5, effort=166.42) inflate the average metric values. Nevertheless, the value ranges for the inferred MRs are narrow and typically favour more compact formulations. For example, in words, the inferred MRs have lower average tree size (16.5 vs 12) and effort (93.64 vs 67.5).

These results suggest that the assistant can infer MRs with a human-plausible complexity, although often tends towards simpler contexts. In addition, in the data centres case study, the assistant generated an MR identical to one from the original work, and two others achieved approximately 70% similarity. This confirms that the assistant can recover human-designed MRs (or close variants) beyond exhibiting comparable complexity statistics.

Answering RQ1. We can answer RQ1 by stating that our proposal is effective at deriving MRs across the domains considered. The ablation study further shows that the structure of the prompt plays a key role in steering the inference process towards semantically meaningful MRs. The computational cost of MR inference (RQ1.1) is low for the evaluated domains. Moreover, the assistant is able to generate MRs that are diverse (RQ1.2), have slightly lower complexity than human-designed rules (RQ1.3), and occasionally reproduce human-designed rules.

5.3. RQ2: Effectiveness in follow-up generation

Experiment design. In this experiment, we built random source test case models, and then generated up to five follow-ups per source, using the 12 MRs in Table 2 (cf. https://github.com/AI4MT/AI4MT-experiment). In some cases, we reduced the number of follow-ups to one, when the formulation of the MR only allowed one follow-up per source test case.

We created source test cases of diverse sizes, ranging from small to large, as Table 5 shows. We established five sizes per case study, and generated between 4 and 6 source test cases for each size. As an exception, the source test cases for Words contain just one object: the word, made of a list of symbols.

The follow-up generation task defined an improvement pipeline with two steps of evaluation and fixing tasks. The first step checked the syntactic well-formedness of the follow-up generated, i.e., that the follow-up was a valid XMI model conformant to the input meta-model. The second step checked that the generated model was a valid follow-up, i.e., it satisfied the left-hand side of the MR, and did not modify elements of the source test case not affected by that left-hand side. When an evaluation task failed, the fixing task was delegated to an LLM passing the erroneous follow-up and a description of the error. Each evaluation and fixing task was iterated up to three times.

Overall, the evaluation aimed at generating 500 follow-ups for the data centres case study (20 source test cases × 5 MRs × 5 follow-ups each), 300 for autonomous vehicles (20 source test cases × 3 MRs × 5 follow-ups each), 60 for automata (30 source test cases × 2 MRs × 1 follow-up each), and 60 for words (30 source test cases × 2 MRs × 1 follow-up each). Since we used five LLMs (*gpt-5-nano*, *o4-mini*, *gpt-4o-mini*, *gpt-4.1-mini*, and *gpt-5-mini*), we aimed at generating 4600 follow-ups overall.

Experiment results. Table 6 shows the results of the follow-up generation. The numbers depict the percentage of correct follow-ups generated for each MR, with each LLM tested. The columns labelled as *Total* aggregate the results for each case study.

We can observe that the assistive task generally obtained good results, since for every MR, at least one LLM correctly generated the number of requested follow-ups. The case study with the best results was autonomous vehicles (an average effectiveness of 95.4% for all LLMs), followed by data centres (87.24%), words (83.3%) and automata (78.6%). This may be attributed to the fact that automata have the inputs with the most complex structure, with transitions connecting three different objects (the source and target states, and the symbol). This leads to the most intricate models to generate.

Next, we analyse in further detail the effects of the improvement loops, the source test case size in correctness and computational cost, and the LLM.

RQ2.1. Effect of the improvement loops. We focus just on the "semantic" loop (i.e., checking if the generated model is a follow-up), as it had a much bigger impact on the overall effectiveness than the syntactic loop. This is so as, most of the times, the generated model was correct XMI, so the XMI fixer task was seldom necessary. Specifically, the XMI fixer was triggered for a 1.3% of the generated follow-ups, and of those, a 91.8% was for *gpt-5-nano*, a 6.6% for *gpt-4o-mini*, and a 1.6% for *o4-mini*.

Table 7 shows the effect of the semantic loops: all case studies and LLMs benefited from them. Using loops resulted in substantial

Table 5
Size of source test case models.

Case study	Source TCs	Objects			Attr. values			References		
		avg	min	max	avg	min	max	avg	min	max
Data Centres	20	71.80	7	203	105.53	8	302	70.80	6	202
Autonomous Vehicles	20	37.65	6	147	89.30	26	308	35.65	4	145
Automata	30	55.20	9	153	56.20	10	154	158.60	20	452
Words	30	1.00	1	1	22.53	4	62	0.00	0	0

Table 6
Total effectiveness of follow-up generation (numbers depict percentage of correctness).

	Data Centres						Autonomous Vehicles				Automata			Words		
LLM	MR1	MR2	MR3	MR4	MR5	Total	MR6	MR7	MR8	Total	MR9	MR10	Total	MR11	MR12	Total
gpt-5-nano	77	73	85	74	77	**77.20**	85	86	83	**84.67**	80	90	**85**	58.33	91.67	**75**
o4-mini	99	100	100	100	100	**99.80**	99	100	99	**99.34**	100	100	**100**	100	100	**100**
gpt-4o-mini	75	21	72	85	66	**63.80**	99	94	93	**95.34**	10	60	**35**	41.67	66.67	**54.17**
gpt-4.1-mini	97	99	96	99	87	**95.60**	100	100	95	**98.34**	70	76.67	**73.34**	100	91.67	**95.84**
gpt-5-mini	100	100	99	100	100	**99.80**	100	98	100	**99.34**	100	100	**100**	100	83.34	**91.67**
Average						**87.24**				**95.4**			**78.6**			**83.3**

Table 7
Effect of improvement loops. Column *Base* shows the correctness percentage obtained without improvement. Column *It. i* shows the percentage points improved after applying the *i*-th improvement iteration, up to three iterations. The biggest gains in each iteration are marked in green.

	Data Centres				Autonomous Vehicles				Automata				Words			
LLM	Base	It. 1	It. 2	It. 3	Base	It. 1	It. 2	It. 3	Base	It. 1	It. 2	It. 3	Base	It. 1	It. 2	It. 3
gpt-5-nano	72.7%	3	4.3	0.2	60.0%	22.4	2.3	0	71.7%	13.3	0	0	68.4%	1.6	0	1.7
o4-mini	99.0%	0	0.7	0	98.0%	1	0.4	0	96.7%	3.3	0	0	96.7%	3.3	0	0
gpt-4o-mini	0.0%	38.4	13.6	12	34.4%	42	16.6	2.4	33.4%	1.6	0	0	36.7%	8.3	3.4	0
gpt-4.1-mini	83.0%	9.2	3.8	0.4	67.7%	26.7	4	0	71.7%	1.7	0	0	80.0%	3.4	1.6	0
gpt-5-mini	99.2%	0	0.5	0.2	98.0%	1	0	0.4	96.7%	3.3	0	0	95.0%	0	0	0

improvements of the correctness percentages, especially for *gpt-4o-mini* with data centres (increasing from 0% to 64%) and autonomous vehicles (from 34.4% to 95.4%). The case study that benefited the most was autonomous vehicles (up by 119.2 percentual points), followed by data centres (up by 86.3 percentual points), words (up by 23.3 percentual points) and automata (up by 23.2 percentual points).

To facilitate comprehension, Fig. 7 represents the increase in correctness percentages by improvement iterations using stacked bar charts, for each LLM and case study. On average, the iteration that achieved the greatest increase in correctness percentage (i.e., fixed the most errors) was the first one, with 58.6% for data centres, 78.1% for autonomous vehicles, 100% for automata, and 71.2% for words. Conversely, the iteration that corrected the least number of defects was the third one, being 14.8% for data centres, 2.35% for autonomous vehicles, 0% for automata, and 7.3% for words. These numbers were calculated by dividing the percentage improvements in iteration i by the total percentage improvements across all iterations, aggregated over all LLMs.

Table 8 shows the percentage of fixes that did not succeed in each iteration, for each LLM. The last column shows the overall percentage of unsuccessful fixes per LLM. An unsuccessful fix can be seen as wasted time, and hence, lower values are better. For example, in the first improvement iteration of data centres, 88.89% of fixes failed for *gpt-5-nano*. The last two rows in the table report aggregated percentages per iteration and case study (the last row). These data reveal interesting aspects. First, most fixes succeed in the first iteration (which is also observed in Table 7), and errors become more difficult to fix in subsequent iterations. This is observed in the "Total %" row, as percentages tend to increase across iterations. Second, the most difficult case studies to fix were automata and words, with unsuccessful rates of 93.20% and 91.95%. Finally, the LLM with the lowest rate of unsuccessful fixes was *gpt-4.1-mini*.

RQ2.2. Effect of source test case size in correctness. To measure the effect of the source test case size on the task effectiveness, we created source test case models in five categories of increasing sizes (cf. Table 5), with equal number of models in each category.

Table 9 shows the average degradation in correctness rates, as measured by the difference in percentage points between the smallest source test cases and the biggest ones. For example, the performance of *gpt-5-nano* in the data centres case study decreased from 100% to 14% for the biggest source test cases, and hence, the table reports −86 points. We can observe that size impacted most LLMs and domains, since in 60% of the cases, the performance with bigger source test cases was worse. The LLMs with better scalability were *o4-mini* and *gpt-5-mini*, while the least scalable was *gpt-5-nano*.

RQ2.3. Computational cost. Table 10 displays, for each LLM, source test case size and domain, the number of cycles required for syntactic and semantic fixes, and the average time to finish the process (either generating a correct follow-up or an incorrect one). The coloured cells signal the fastest times. We report the number of cycles, since each cycle implies a call to the LLM, increasing the computational cost.

gpt-4.1-mini was the fastest LLM 17 out of 20 times. *o4-mini* and *gpt-5-mini* required the least number of repairs, becoming the fastest in the remaining 3 cases. These cases involved bigger source test cases (sizes 4 and 5), where *gpt-4.1-mini* had to apply a substantial number of repair cycles. The case of *gpt-4o-mini* is interesting, as it has good speed in tokens/s, but generally needs to perform many repair cycles, resulting in overall higher times than other LLMs (even reasoning ones).

Reasoning LLMs (*o4-mini* and *gpt-5-mini*) are generally slow, and hence repair cycles are a high penalty in them. This can be seen in the words case study for *gpt-5-mini*, in which one of the slowest times was with the smallest source test case (size 1), because it needed 6 repairs.

Finally, we observe that execution times tend to increase with source test case size. This is due to two factors: (a) the prompt needs to include a larger source test case and becomes longer, and (b) the problem to solve becomes more difficult, likely requiring more repair cycles that increase the calls to the LLM. Moreover, the prompt to generate the nth follow-up includes the previous n-1 follow-ups to avoid repetition, which also has an effect on the processing time.

RQ2.4. Effect of LLM. Table 11 summarises the performance of the LLMs in the follow-up generation task. It reports the total percentage of correctly generated follow-ups, the number of MRs for which the LLM

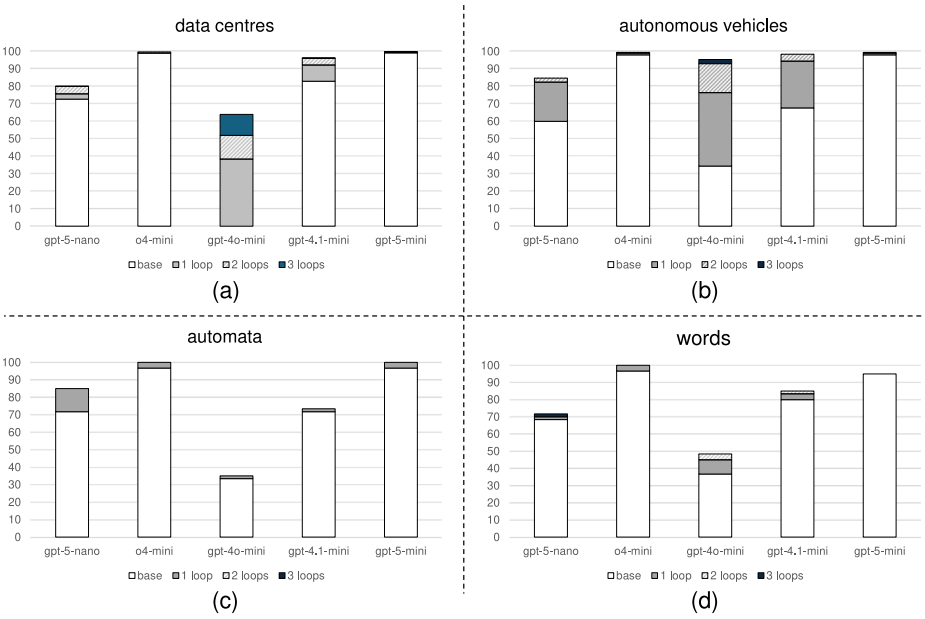


Fig. 7. Correctness percentages after the improvement loops on each LLM and case study.

Table 8
Percentage of unsuccessful repair loops across iterations.

LLM	Data Centres			Autonomous Vehicles			Automata			Words			Over.
	It. 1	It. 2	It. 3	It. 1	It. 2	It. 3	It. 1	It. 2	It. 3	It. 1	It. 2	It. 3	%
gpt-5-nano	88.89	84.56	99.13	44.17	86.79	100.00	52.94	100.00	100.00	94.74	100.00	100.00	82.88
o4-mini	100.00	20.00	100.00	50.00	66.67	100.00	0.00	0.00	0.00	0.00	0.00	0.00	53.84
gpt-4o-mini	63.00	75.56	75.21	36.04	29.58	66.67	97.50	100.00	100.00	86.84	93.94	93.55	67.09
gpt-4.1-mini	47.47	51.06	91.67	17.53	29.41	100.00	94.12	100.00	100.00	83.33	90.00	88.89	52.84
gpt-5-mini	100.00	40.00	50.00	50.00	100.00	66.67	0.00	0.00	0.00	100.00	100.00	100.00	71.42
Total %	66.67	74.80	83.42	34.50	52.38	89.61	82.05	100	100	86.48	95.31	95.08	69.24
Aggregated		73.03			45.07			93.20			91.95		

Table 9
Correctness performance degradation when increasing source test case size.

LLM	Data Centres	Aut. Vehicles	Automata	Words
gpt-5-nano	−86	−50	0	−50
o4-mini	−1	−3.33	0	0
gpt-4o-mini	−33	−8.33	−16.67	−50
gpt-4.1-mini	−8	0	−8.34	−33.33
gpt-5-mini	0	0	0	8.33

correctly generated all follow-ups (with 12 the maximum possible), the percentage of different follow-ups, the number of follow-ups requiring syntactic fixes,[5] the correctness percentage points increased due to the improvement loops (counted from 0 to 100 in each case study), the correctness degradation when increasing the source test case size, the number of times the LLM the fastest (across the 4 case studies and 5 source test case sizes), and the time degradation in seconds with source test case size.

We observe that the best overall LLMs for this task were *o4-mini* and *gpt-5-mini*, as they achieved correctness percentages of 99.61% and 99.22%. They also correctly generated all follow-ups for 8 and 9 MRs out of 12 (66.7% and 75%), and most or all generated follow-ups were different (99.9% and 100%). Moreover, they required very few syntactic or semantic fixes, and hence benefitted less from the improvements loops. Instead, the worst performing LLM (*gpt-4o-mini*) was the one that benefitted the most from improvement loops, increasing 138.3 percentage points the correctness of the generated follow-ups across all case studies, or a 34.57%. Column *Correct. degrad.* calculates the task performance degradation as the difference of performance between the smallest and biggest source test cases. We observe a severe degradation in *gpt-5-nano* (a 46.5% decrease), and to a lesser extent, in *gpt-4o-mini* (−27%) and *gpt-4.1-mini* (−12.41%). Instead, the other two LLMs are more stable with size, with *gpt-5-mini* even showing a small percentual increase. Regarding time, *gpt-4.1-mini* was generally the fastest LLM. However, reasoning LLMs (*o4-mini* and *gpt-5-mini*) showed better time degradation with size than the other LLMs. Hence, considering all factors, using a fast LLM like *gpt-4.1-mini* is sufficient for small source test cases, whereas a reasoning LLM may pay off for large source test cases or complex domains. Combined pipelines that use a stronger (but slower) LLM after an unsuccessful generation by a weaker (but faster) LLM may be investigated in future work.

Answering RQ2. Our assistance architecture was able to generate follow-ups for all case studies with high success (cf. Table 6), where *o4-mini* and *gpt-5-mini* exceeded 99% correctness rates (cf. Table 11). Improvement workflows (RQ2.1) helped to improve performance in up to 34.57 points for *gpt-4o-mini*, and the most significant improvement loop was the semantic one, as syntactic fixes were only triggered in 1.3% of follow-ups. The size of the source test cases (RQ2.2) is a

[5] In comparison, Table 10 showed the number of cycles required for syntactic and semantic fixes.

Table 10
Computation cost of follow-up generation (in seconds), across source test case sizes, domains, and LLMs.

Source TC Size	LLM	Data centres			Autonomous Vehicles			Automata			Words		
		Syn. cycles	Sem. cycles	Avg. time	Syn. cycles	Sem. cycles	Avg. time	Syn. cycles	Sem. cycles	Avg. time	Syn. cycles	Sem. cycles	Avg. time
size 1	gpt-5-nano	0	15	20.63	3	17	28.05	0	8	58.12	3	4	22.88
	o4-mini	0	8	12.18	0	0	20.24	0	0	20.94	0	0	11.45
	gpt-4o-mini	0	188	12.63	0	57	10.08	0	22	25.10	0	10	4.32
	gpt-4.1-mini	0	23	2.78	0	29	7.12	0	9	8.47	0	4	2.05
	gpt-5-mini	0	9	14.23	0	1	20.73	0	0	20.80	0	6	15.43
size 2	gpt-5-nano	0	9	27.45	15	34	34.24	0	8	62.66	6	15	32.04
	o4-mini	0	0	18.65	0	0	18.16	0	0	18.38	0	0	11.06
	gpt-4o-mini	0	201	28.73	0	61	11.33	0	21	28.16	0	28	10.39
	gpt-4.1-mini	0	19	6.83	0	42	10.24	0	6	8.61	0	1	2.25
	gpt-5-mini	0	0	24.29	0	3	18.48	0	0	24.88	0	0	12.81
size 3	gpt-5-nano	21	39	43.21	0	22	32.01	4	4	47.83	0	6	25.39
	o4-mini	0	0	23.87	0	0	21.61	0	0	21.01	0	0	12.82
	gpt-4o-mini	0	201	44.39	0	53	13.69	0	27	60.69	0	15	8.84
	gpt-4.1-mini	0	30	11.81	0	20	7.73	0	6	12.19	0	6	3.06
	gpt-5-mini	0	0	32.97	0	8	21.45	0	0	29.05	0	0	14.49
size 4	gpt-5-nano	30	68	44.69	6	36	51.80	3	7	86.95	0	6	33.59
	o4-mini	0	0	29.65	0	0	29.85	0	0	27.86	0	0	20.05
	gpt-4o-mini	0	227	88.19	0	60	36.80	0	21	77.70	0	21	22.19
	gpt-4.1-mini	0	37	26.77	0	12	16.08	0	15	42.33	0	5	7.35
	gpt-5-mini	0	3	44.90	0	0	31.76	0	0	40.46	0	0	16.57
size 5	gpt-5-nano	15	273	157.16	42	110	107.10	4	8	112.27	12	24	246.48
	o4-mini	0	3	42.41	3	11	41.16	0	2	69.33	0	2	24.63
	gpt-4o-mini	0	236	208.62	0	58	59.11	9	27	237.00	0	28	40.10
	gpt-4.1-mini	0	61	57.11	0	16	33.74	0	13	115.86	0	15	16.87
	gpt-5-mini	0	0	73.56	0	0	48.35	0	2	68.58	0	3	25.71

Table 11
Performance of LLMs in follow-up generation.

LLM	Total %	#Perfect MRs	Diff %	#Syn. fixes	Loop improv.	Correct. degrad.	Times fastest	Time degrad. (s)
gpt-5-nano	81.47%	0	95.4%	56	12.2%	−46.5%	0	123.33
o4-mini	99.61%	8	99.9%	1	2.17%	−1.08%	2	28.18
gpt-4o-mini	70.73%	0	99.7%	4	34.57%	−27%	0	123.18
gpt-4.1-mini	95.54%	3	99.4%	0	12.7%	−12.41%	17	50.79
gpt-5-mini	99.22%	9	100%	0	1.35%	+2.08%	1	36.25

significant factor for correctness, impacting most LLMs and case studies, especially for *gpt-5-nano*. Computation cost (RQ2.3) increases with source test case size and is impacted by the number of repair cycles required. In practice, faster (i.e., non-reasoning) LLMs are advisable for smaller test cases, while reasoning LLMs may pay off for larger cases to avoid the cost of repair cycles. The selection of the LLM (RQ2.4) has also a high impact, where LLMs oriented to reasoning (*o4-mini* and *gpt-5-mini*) achieved better results and scalability.

5.4. Threats to validity

Next, we discuss threats affecting the validity of the experiment results, as well as mitigating strategies implemented.

Internal validity. For RQ2, we could have considered more follow-ups, and larger ones. Still, we used a sufficiently large number of source test cases (100) with substantial sizes, and attempted to generate 920 follow-ups per LLM. This was enough to reveal the effects of source test case size in RQ2.2. Moreover, the *small scope hypothesis* in testing states that if a fault exists in a program, it is likely to be revealed within a small input scope, rather than only in large or complex ones [42].

External validity. Regarding generalisability, we chose case studies from different domains. While more confidence could be obtained using more domains, the used ones were carefully chosen due to their very distinct nature. We have evaluated our architecture on two MT tasks: MR inference and follow-up generation. We chose these tasks because they are central to MT. However, we would need to evaluate the effectiveness of the architecture with the other 10 tasks in the taxonomy in future work.

Construct validity. When designing experiments with LLMs, there may be risk of *data contamination* [43]. This occurs when the training data of the LLM have seen, even partially, the information used in the tasks the LLM is asked to perform, as this may cause artificially good results. To mitigate this threat, we created the GOTTEN specifications for autonomous vehicles, automata and words specifically for this work, and only the data centres case study was taken from [6]. Still, RQ1 involved the creation of new MRs (different from those in [6]), while RQ2 involved the LLM-based generation of follow-ups.

For RQ1, MR inference was assessed in terms of syntactic and semantic validity, diversity, complexity, and computational cost, but not in terms of fault detection capability. Hence, while these measures provide evidence that the inferred MRs are meaningful and useable artefacts, they do not yet allow concluding that they are sufficiently effective at revealing real faults. In practice, this means that the inferred MRs may reduce the effort of MR elicitation, but their fault-revealing effectiveness still requires dedicated assessment, for instance through mutation testing.

For RQ2, we used the 12 MRs of Table 2, since they were defined and validated by humans. In RQ1, our system generated 26 new MRs. Using all of them in RQ2 would have increased the computational time and cost budget of the experiment. Since filtering some of them would have raised concerns on the criteria used for selection and could introduce bias, we used the original MRs.

6. Conclusions and future work

This paper has proposed an architecture to automate the tasks involved in MT via LLMs, focusing on MT inference and follow-up generation. The architecture combines both AI tasks and MT framework

tasks, and supports the quality improvement of AI task results via workflows of loops with evaluation and fixing tasks. We have realised the architecture as a Java library, and integrated it into the Gotten MT framework. We have demonstrated the usefulness of two tasks (MR inference and follow-up generation) analysing the effect on effectiveness of the prompt structure, the improvement loops, the source test case size, and the LLM used. Our work opens the door to a better understanding of LLM assistance in key MT tasks.

In the future, we will perform detailed evaluations of the effectiveness of the remaining AI-based MT tasks in our taxonomy, over other domains, and enabled by other LLMs. We will perform more focussed evaluations, e.g., on the fault detection capabilities of automatically inferred MRs, and user studies on the actual test reduction effort that AI automation brings. We also plan to decouple even more the AI MT assistance framework by the use of services and the Model Context Protocol (MCP).[6]

CRediT authorship contribution statement

Pablo C. Cañizares: Writing – original draft, Software, Methodology, Investigation, Conceptualization. **Pablo Gómez-Abajo:** Writing – original draft, Software, Investigation, Funding acquisition. **Esther Guerra:** Writing – original draft, Methodology, Investigation, Funding acquisition, Conceptualization. **Juan de Lara:** Writing – original draft, Software, Methodology, Investigation, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Juan de Lara reports financial support was provided by Spanish Ministry of Science, Innovation and Universities. Pablo Gomez-Abajo reports financial support was provided by Community of Madrid. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

Work funded by the Spanish MICINN with projects PID2021-122270OB-I00, PID2024-155231OB-I00 and RED2022-134647-T, and by the Community of Madrid's PRICIT programme with project 2025/00097/001.

Data availability

Data available at: https://github.com/AI4MT/AI4MT-experiment.

References

[1] E.T. Barr, M. Harman, P. McMinn, M. Shahbaz, S. Yoo, The oracle problem in software testing: A survey, IEEE Trans. Softw. Eng. 41 (5) (2015) 507–525.
[2] S. Segura, G. Fraser, A.B. Sánchez, A.R. Cortés, A survey on metamorphic testing, IEEE Trans. Softw. Eng. 42 (9) (2016) 805–824.
[3] E.J. Weyuker, On testing non-testable programs, Comput. J. 25 (4) (1982) 465–470.
[4] T.Y. Chen, S.C. Cheung, S.M. Yiu, Metamorphic Testing: A New Approach for Generating Next Test Cases, Tech. rep., HKUST-CS98-01, 1998.
[5] S. Segura, D. Towey, Z.Q. Zhou, T.Y. Chen, Metamorphic testing: Testing the untestable, IEEE Softw. 37 (3) (2020) 46–53.
[6] P. Gómez-Abajo, P.C. Cañizares, A.N. nez, E. Guerra, J. de Lara, Automated engineering of domain-specific metamorphic testing environments, Inf. Softw. Technol. 157 (2023) 107164.
[7] S. Speth, A. Pretschner, GeMTest: A general metamorphic testing framework, in: ICSE, IEEE, 2025, pp. 41–44.
[8] J. Wang, et al., Software testing with large language models: Survey, landscape, and vision, IEEE Trans. Softw. Eng. (ISSN: 0098-5589) 50 (4) (2024) 911–936.
[9] V. Guilherme, A. Vincenzi, An initial investigation of ChatGPT unit test generation capability, in: SAST, ACM, 2023, pp. 15–24.
[10] M. Schäfer, S. Nadi, A. Eghbali, F. Tip, An empirical evaluation of using large language models for automated unit test generation, IEEE Trans. Softw. Eng. (2023).
[11] M. Tufano, D. Drain, A. Svyatkovskiy, N. Sundaresan, Generating accurate assert statements for unit test cases using pretrained transformers, in: AST@ICSE, ACM, 2022, pp. 54–64.
[12] P. Mahbub, O. Shuvo, M.M. Rahman, Explaining software bugs leveraging code structures in neural machine translation, in: ICSE, IEEE, 2023, pp. 640–652.
[13] R. Li, et al., Metamorphic relation generation: State of the art and research directions, ACM Trans. Softw. Eng. Methodol. 34 (5) (2025) 149:1–149:25.
[14] Z.Q. Zhou, T.H. Tse, F.-C. Kuo, T.Y. Chen, Automated functional testing of web search engines in the absence of an oracle, Tech. Rep. TR-2007-06, The University of Hong Kong, 2007.
[15] J. Ayerdi, V. Terragni, G. Jahangirova, A. Arrieta, P. Tonella, GenMorph: Automatically generating metamorphic relations via genetic programming, IEEE Trans. Softw. Eng. 50 (7) (2024) 1888–1900.
[16] C. Tsigkanos, P. Rani, S. Müller, T. Kehrer, Variable discovery with large language models for metamorphic testing of scientific software, in: ICCS, in: LNCS, vol. 14073, Springer, 2023, pp. 321–335.
[17] Y. Zhang, D. Towey, M. Pike, Automated metamorphic-relation generation with ChatGPT: An experience report, in: COMPSAC, IEEE, 2023, pp. 1780–1785.
[18] Y. Zhang, et al., Enhancing autonomous driving simulations: A hybrid metamorphic testing framework with metamorphic relations generated by GPT, Inf. Softw. Technol. 187 (2025) 107828.
[19] Q.-H. Luu, H. Liu, T.Y. Chen, Can ChatGPT advance software testing intelligence? An experience report on metamorphic testing, 2023, arXiv:2310.19204.
[20] S.Y. Shin, F. Pastore, D. Bianculli, A. Baicoianu, Towards generating executable metamorphic relations using large language models, in: QUATIC, in: CCIS, vol. 2178, Springer, 2024, pp. 126–141.
[21] S. Pedram, Y. Labiche, Using category partition to detect metamorphic relations, in: MET@ICSE 2024, ACM, 2024, pp. 10–17.
[22] C. Xu, V. Terragni, H. Zhu, J. Wu, S.-C. Cheung, MR-Scout: Automated synthesis of metamorphic relations from existing test cases, ACM Trans. Softw. Eng. Methodol. 33 (6) (2024) 150.
[23] H. Liu, X. Liu, T.Y. Chen, A new method for constructing metamorphic relations, in: QSIC, IEEE, 2012, pp. 59–68.
[24] A. Arrieta, On the cost-effectiveness of composite metamorphic relations for testing deep learning systems, in: MET@ICSE 2022, ACM, 2022, pp. 42–47.
[25] Z. Xiang, H. Wu, F. Yu, A genetic algorithm-based approach for composite metamorphic relations construction, Inf. 10 (12) (2019) 392.
[26] K. Qiu, Z. Zheng, T.Y. Chen, P. Poon, Theoretical and empirical analyses of the effectiveness of metamorphic relation composition, IEEE Trans. Softw. Eng. 48 (3) (2022) 1001–1017.
[27] P.Y.P. Chan, J. Keung, Z. Yang, Effectiveness of symmetric metamorphic relations on validating the stability of code generation LLM, J. Syst. Softw. 222 (2025) 112330.
[28] S. Cho, S. Ruberto, V. Terragni, Metamorphic testing of large language models for natural language processing, in: ICSME, 2025.
[29] G. Guo, A. Aleti, N. Neelofar, C. Tantithamthavorn, MORTAR: Metamorphic multi-turn testing for LLM-based dialogue systems, 2024, arXiv:2412.15557.
[30] S. Hyun, M. Guo, M.A. Babar, METAL: Metamorphic testing framework for analyzing large-language model qualities, in: ICST, IEEE, 2024, pp. 117–128.
[31] G. Batra, J. Sengupta, An efficient metamorphic testing technique using genetic algorithm, in: ICISTM, in: CCIS, vol. 141, Springer, 2011, pp. 180–188.
[32] P. Godefroid, A. Kiezun, M.Y. Levin, Grammar-based whitebox fuzzing, in: PLDI, ACM, 2008, pp. 206–215.
[33] V. Atlidakis, P. Godefroid, M. Polishchuk, RESTler: Stateful REST API fuzzing, in: ICSE, IEEE / ACM, 2019, pp. 748–758.
[34] K. Claessen, J. Hughes, QuickCheck: A lightweight tool for random testing of haskell programs, in: ICFP, ACM, 2000, pp. 268–279.
[35] OCL, http://www.omg.org/spec/OCL/.
[36] P.C. Cañizares, P. Gómez-Abajo, A. Núñez, E. Guerra, J. de Lara, New ideas: Automated engineering of metamorphic testing environments for domain-specific languages, in: SLE, ACM, 2021, pp. 49–54.
[37] P. Gómez-Abajo, P.C. Cañizares, A. Núñez, E. Guerra, J. de Lara, Gotten: A model-driven solution to engineer domain-specific metamorphic testing environments, in: MODELS-C, IEEE, 2023, pp. 65–69.

[6] https://modelcontextprotocol.io/

[38] J. Ayerdi, et al., Performance-driven metamorphic testing of cyber-physical systems, IEEE Trans. Reliab. 72 (2) (2023) 827–845.
[39] MathWorks Student Competitions Team, MathWorks/vehicle-purepursuit, 2026, https://github.com/mathworks/vehicle-pure-pursuit. (Last Access).
[40] Z. Li, J. Chen, X. Xie, T.Y. Chen, S. Cai, SAME: A similarity analysis method for evaluating metamorphic relations in testing AI systems, Inf. Softw. Technol. (2026) 108069.
[41] T. Hariprasad, G. Vidhyagaran, K. Seenu, C. Thirumalai, Software complexity analysis using halstead metrics, in: ICEI, IEEE, 2017, pp. 1109–1113.
[42] D. Jackson, C. Damon, Elements of style: Analyzing a software design feature with a counterexample detector, IEEE Trans. Softw. Eng. 22 (7) (1996) 484–495.
[43] O. Sainz, et al., NLP evaluation in trouble: On the need to measure LLM data contamination for each benchmark, in: EMNLP (Findings), ACL, 2023, pp. 10776–10787.